```
clc;
clear;
close all;
```

PARAMETERS

```
A = 1;
f = 50;
Fs = 1000;
N = 1000;

n = 0:N-1;
x = A*sin(2*pi*f*n/Fs);

B = 8;
bits = [4 8 12 16];
```

8-BIT QUANTIZATION

```
D = 2*A/2^B;

xr = D*round(x/D); % Rounding
xt = D*fix(x/D);   % Truncation

er = x-xr; % Rounding error
et = x-xt; % Truncation error

SQr = 10*log10(mean(x.^2)/mean(er.^2));
SQt = 10*log10(mean(x.^2)/mean(et.^2));
```

INPUT / OUTPUT QUANTIZATION

```
xq = xr;
y = x;
yq = D*round(y/D);
eo = y-yq;

SQout = 10*log10(mean(y.^2)/mean(eo.^2));
```

WORD-LENGTH COMPARISON

```
for k = 1:length(bits)
    b = bits(k);
    d = 2*A/2^b;

    qr = d*round(x/d);
    qt = d*fix(x/d);

    er1 = x-qr;
```

```
    et1 = x-qt;

    step(k) = d;
    sqnrR(k) = 10*log10(mean(x.^2)/mean(er1.^2));
    sqnrT(k) = 10*log10(mean(x.^2)/mean(et1.^2));
    maxER(k) = max(abs(er1));
    maxET(k) = max(abs(et1));
end
```

RESULTS

```
fprintf('\n========== QUANTIZATION RESULTS ==========\n');
fprintf('Amplitude = %.1f | Frequency = %d Hz | Fs = %d Hz\n', A, f, Fs);
fprintf('Samples = %d | Main word length = %d bits\n', N, B);
fprintf('8-bit Step Size = %.7f\n', D);
fprintf('Rounding SQNR = %.4f dB\n', SQr);
fprintf('Truncation SQNR = %.4f dB\n', SQt);
fprintf('Output SQNR = %.4f dB\n', SQout);
fprintf('Max Rounding Error = %.6f\n', max(abs(er)));
fprintf('Max Truncation Error = %.6f\n', max(abs(et)));
```

```
========== QUANTIZATION RESULTS ==========
Amplitude = 1.0 | Frequency = 50 Hz | Fs = 1000 Hz
Samples = 1000 | Main word length = 8 bits
8-bit Step Size = 0.0078125
Rounding SQNR = 48.9339 dB
Truncation SQNR = 45.2931 dB
Output SQNR = 48.9339 dB
Max Rounding Error = 0.003483
Max Truncation Error = 0.005744
```

FIRST 10 SAMPLES

```
T1 = table(n(1:10)', x(1:10)', xr(1:10)', ...
    er(1:10)', xt(1:10)', et(1:10)', ...
    'VariableNames', {'n','Original','Rounded', ...
    'RoundError','Truncated','TruncError'});

disp(T1);
```

n	Original	Rounded	RoundError	Truncated	TruncError
0	0	0	0	0	0
1	0.30902	0.3125	-0.003483	0.30469	0.0043295
2	0.58779	0.58594	0.0018478	0.58594	0.0018478
3	0.80902	0.8125	-0.003483	0.80469	0.0043295
4	0.95106	0.95312	-0.0020685	0.94531	0.005744
5	1	1	0	1	0
6	0.95106	0.95312	-0.0020685	0.94531	0.005744
7	0.80902	0.8125	-0.003483	0.80469	0.0043295
8	0.58779	0.58594	0.0018478	0.58594	0.0018478

```
    9     0.30902       0.3125      -0.003483      0.30469      0.0043295
```

WORD-LENGTH TABLE

```
T2 = table(bits', step', sqnrR', sqnrT', maxER', maxET', ...
    'VariableNames', {'Bits','StepSize','SQNR_Round_dB', ...
    'SQNR_Trunc_dB','MaxError_Round','MaxError_Trunc'});

disp(T2);
```

```
    Bits      StepSize      SQNR_Round_dB     SQNR_Trunc_dB
MaxError_Round     MaxError_Trunc
    ____     __________     _____________     _____________
_____________     _____________

      4           0.125       23.667            20.871
0.059017           0.087785
      8       0.0078125       48.934            45.293
0.003483           0.005744
     12      0.00048828       78.811            65.891
0.00011536         0.00042324
     16      3.0518e-05       99.697            91.599
1.382e-05          2.6516e-05
```

OUTPUT PARAMETER TABLE

```
Parameter = {
    'Quantized signal'
    'Quantization error'
    'SQNR'
    'Word-length comparison'
};

Observation = {
    sprintf('8-bit rounding, Delta = %.7f', D)
    sprintf('Max rounding error = %.6f', max(abs(er)))
    sprintf('Rounding = %.4f dB; Truncation = %.4f dB', SQr, SQt)
    'SQNR increases as word length increases'
};

Unit = {
    'Amplitude'
    'Amplitude'
    'dB'
    'Bits / dB'
};

disp(table(Parameter, Observation, Unit));
```

```
            Parameter
Observation                              Unit
```

```
    {'Quantized signal'      }    {'8-bit rounding, Delta =
0.0078125'             }    {'Amplitude'}
    {'Quantization error'    }    {'Max rounding error =
0.003483'                }    {'Amplitude'}
    {'SQNR'                  }    {'Rounding = 48.9339 dB; Truncation =
45.2931 dB'}    {'dB'       }
    {'Word-length comparison'}    {'SQNR increases as word length
increases'       }    {'Bits / dB'}
```

GRAPHS

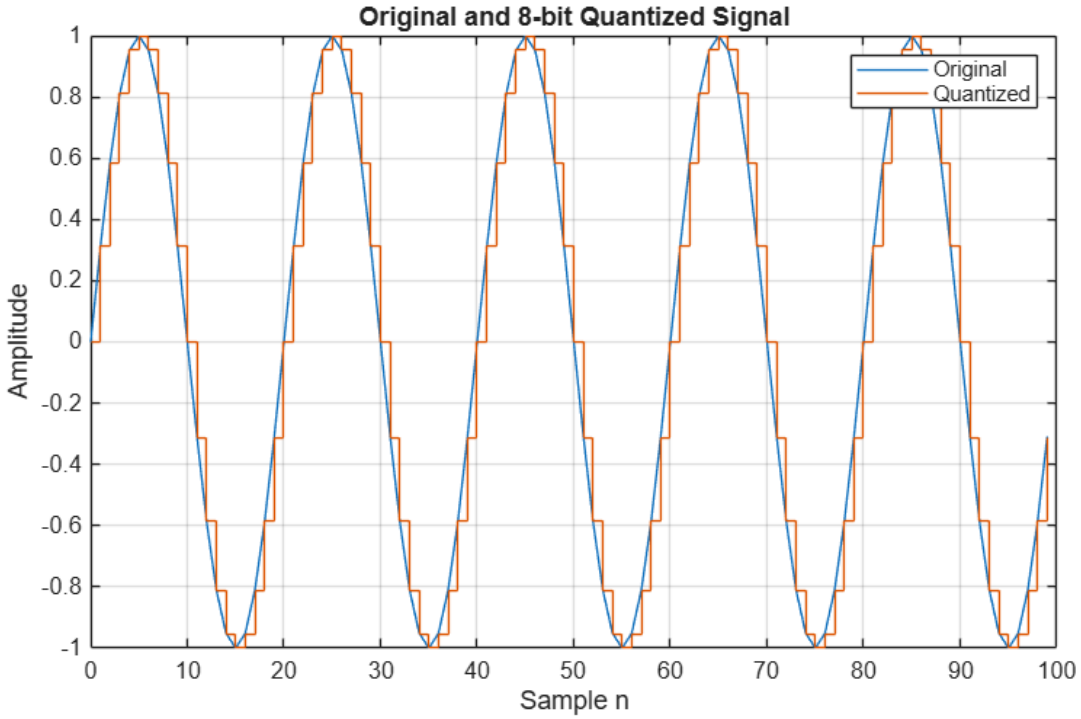
```
figure;
plot(n(1:100), x(1:100));
hold on;
stairs(n(1:100), xr(1:100));
grid on;
xlabel('Sample n');
ylabel('Amplitude');
title('Original and 8-bit Quantized Signal');
legend('Original','Quantized');

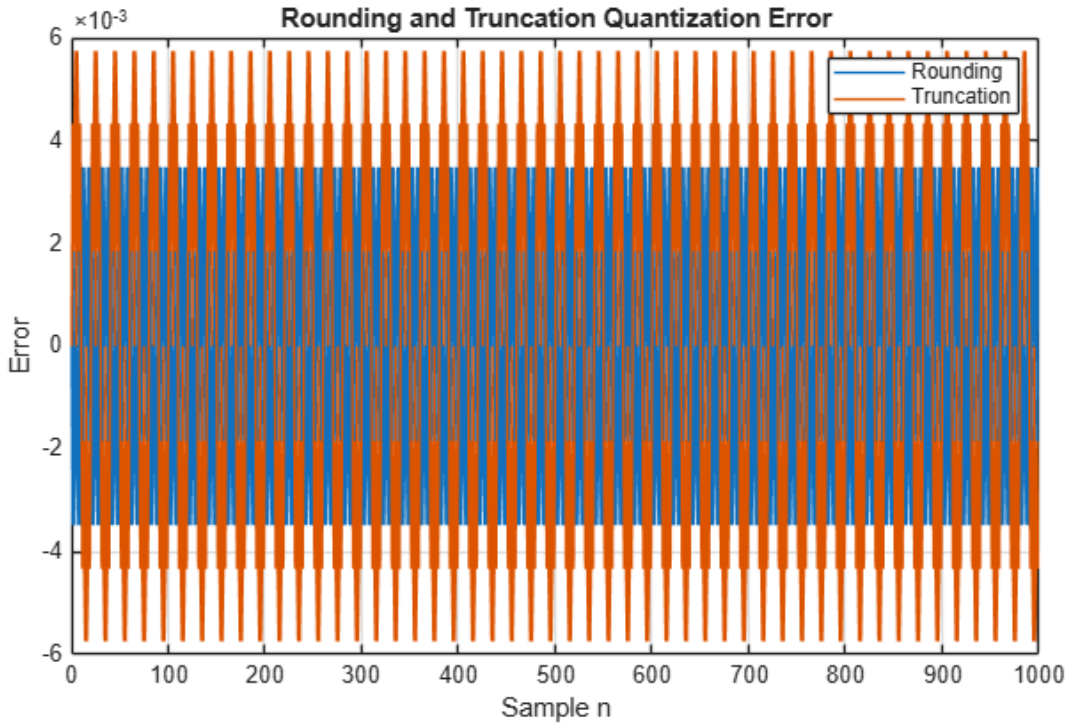
figure;
plot(n, er);
hold on;
plot(n, et);
grid on;
xlabel('Sample n');
ylabel('Error');
title('Rounding and Truncation Quantization Error');
legend('Rounding','Truncation');

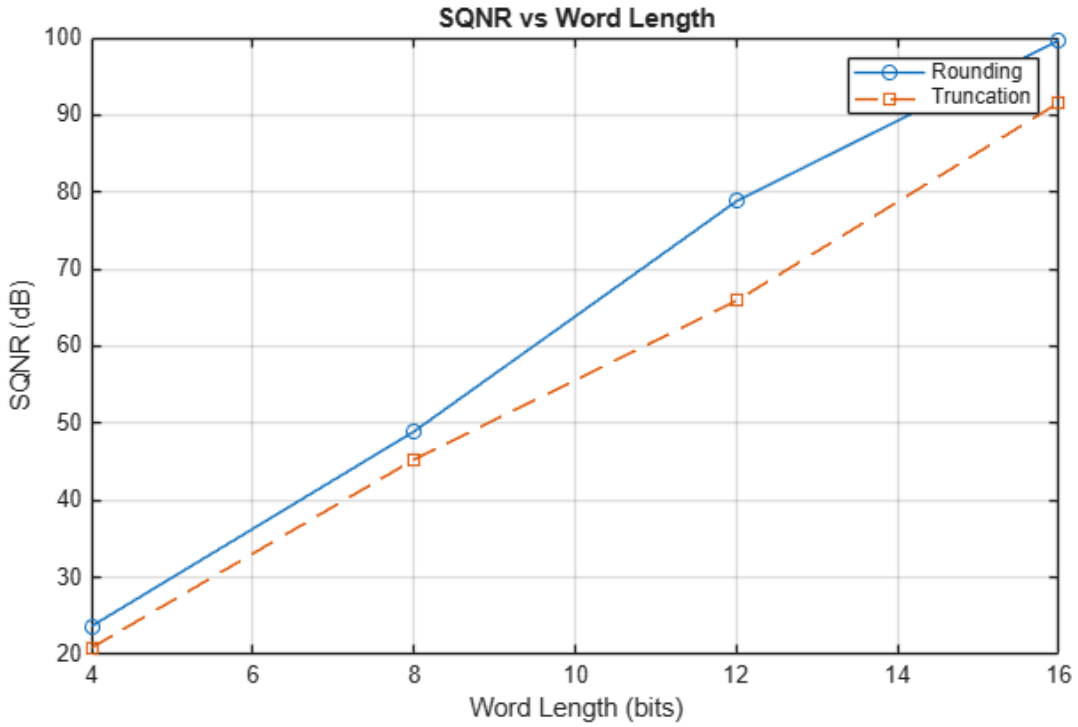
figure;
plot(bits, sqnrR, 'o-');
hold on;
plot(bits, sqnrT, 's--');
grid on;
xlabel('Word Length (bits)');
ylabel('SQNR (dB)');
title('SQNR vs Word Length');
legend('Rounding','Truncation');

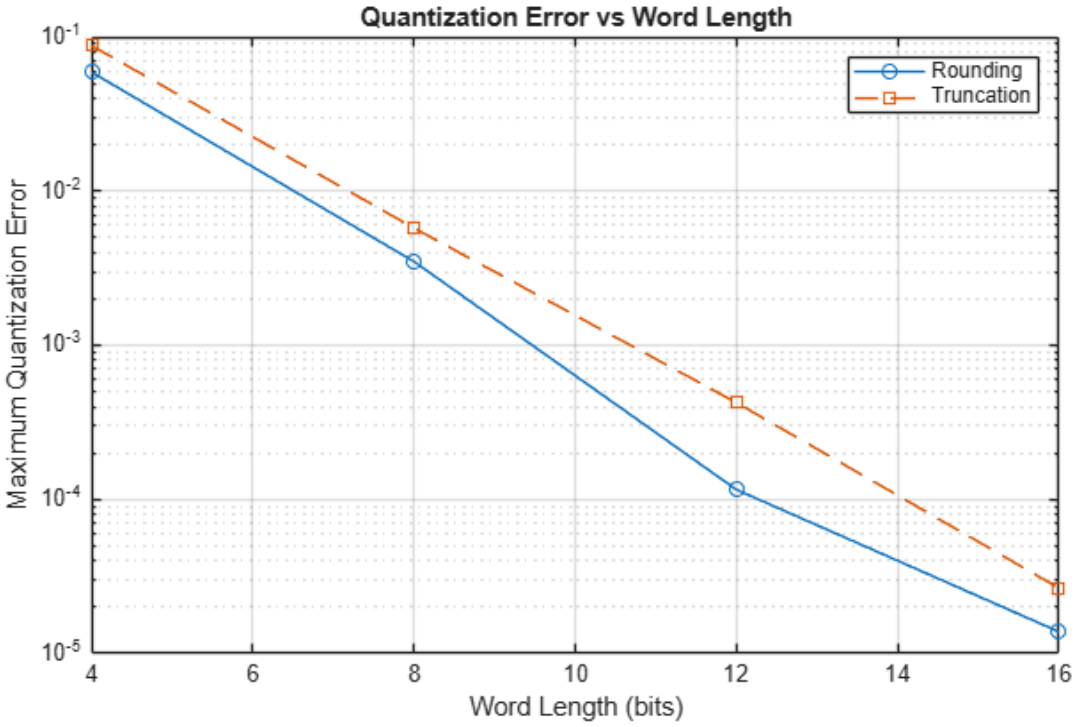
figure;
semilogy(bits, maxER, 'o-');
hold on;
semilogy(bits, maxET, 's--');
grid on;
xlabel('Word Length (bits)');
ylabel('Maximum Quantization Error');
```

```
title('Quantization Error vs Word Length');
legend('Rounding','Truncation');
```









FINAL RESULT

```
fprintf('\n========== FINAL RESULT ==========\n');
fprintf('Uniform quantization using rounding and truncation was
```

```
performed.\n');
fprintf('Quantization error and SQNR were calculated.\n');
fprintf('Increasing word length reduces quantization error and improves
SQNR.\n');
fprintf('Finite word-length effects are therefore verified.\n');
```

```
========== FINAL RESULT ==========
Uniform quantization using rounding and truncation was performed.
Quantization error and SQNR were calculated.
Increasing word length reduces quantization error and improves SQNR.
Finite word-length effects are therefore verified.
```

Published with MATLAB® R2026b